Hafta 5 — Derinlemesine

> ### ⚠️ BU DOSYA SINAV KAPSAMI DIŞINDADIR
>
> Buradaki hiçbir konudan sınavda soru sorulmaz. Bu dosya, "*peki bu nasıl çalışıyor?*" diye soran öğrenci içindir. Sınav kapsamı `H05-ders-notu.md` dosyasıdır.
>
> Dönem projesinde işinize yarayacak birkaç şey de buradadır (bölüm 1, bölüm 3, bölüm 6).

---

1. Kendi Tag Helper'ınızda İleri Adımlar

Ders notu bölüm 5'te `EmailTagHelper`, `FiyatTagHelper`, `BoldTagHelper` ve `ConditionTagHelper`'ı yazdık. Burada dönem projesinde işinize yarayabilecek devamı var. Örneklerin çoğu Microsoft Learn'ün *Author Tag Helpers* sayfasından.

Adları kendiniz seçmek

Ad kuralını beğenmezseniz açıkça yazabilirsiniz:

```
[HtmlTargetElement("para")]                    // <para> olsun
public class FiyatTagHelper : TagHelper
{
    [HtmlAttributeName("tutar")]               // attribute adı "tutar" olsun
    public decimal Deger { get; set; }
}
```

`[HtmlTargetElement]`'e verilen ad **kebab-case** yazılmalıdır: `[HtmlTargetElement("WebsiteInformation")]` yazarsanız `<website-information>` eşleşmez.

Birden çok `[HtmlTargetElement]` **VEYA** demektir; tek bir `[HtmlTargetElement]` içinde ad ve attribute birlikte verilirse **VE** demektir:

```
[HtmlTargetElement("bold")]
[HtmlTargetElement(Attributes = "bold")]          // <bold> YA DA bold attribute'ü

[HtmlTargetElement("bold", Attributes = "bold")]  // yalnızca <bold bold>
```

View'da yalnızca açılış etiketiyle yazılsın istiyorsanız (`<email mail-to="x">`, kapanışsız): `[HtmlTargetElement("email", TagStructure = TagStructure.WithoutEndTag)]`.

`TagHelperOutput`'un ders notunda olmayan kolları

Yazım	Etkisi
`output.TagName = null`	Etiketi **tamamen kaldırır**, yalnızca içeriği bırakır
`output.Content.SetHtmlContent(…)`	İçeriği **ham HTML** olarak yazar (dikkat!)
`output.PreElement.AppendHtml(…)`	Etiketin **önüne** HTML ekler
`output.PostElement.AppendHtml(…)`	Etiketin **arkasına** HTML ekler
`await output.GetChildContentAsync()`	Etiketin içindeki içeriği okur

> ⚠️ `SetHtmlContent` kullanıcıdan gelen veriyle **birleştirilmemelidir**. Hafta 3'te `@Html.Raw` için söylediğimiz XSS uyarısı burada da geçerlidir. `SetContent` ise metni kodlayarak yazar; `₺` işaretinin Ctrl+U'da `₺` görünmesinin sebebi budur.

`ProcessAsync` — etiketin içini okumak

Learn'deki ikinci `EmailTagHelper`, adresi attribute'ten değil etiketin **içinden** alır:

```
public override async Task ProcessAsync(TagHelperContext context, TagHelperOutput output)
{
    output.TagName = "a";
    var icerik = await output.GetChildContentAsync();
    var hedef = icerik.GetContent() + "@contoso.com";
    output.Attributes.SetAttribute("href", "mailto:" + hedef);
    output.Content.SetContent(hedef);
}
```

```
<email>Support</email>   →   <a href="mailto:Support@contoso.com">Support@contoso.com</a>
```

Aynı etikete bağlanan iki Tag Helper varsa hangisinin önce çalışacağını `Order` özelliği belirler (küçük olan önce). Learn'deki `AutoLinker` örneği, sıralama yanlışken bir Tag Helper'ın diğerinin çıktısını nasıl ezdiğini adım adım gösterir.

Var olan bir etiketi zenginleştirmek

Ders notundaki `bold` örneğinin işe yarar bir akrabası:

```
[HtmlTargetElement("a", Attributes = "dis-baglanti")]
public class DisBaglantiTagHelper : TagHelper
{
    public override void Process(TagHelperContext context, TagHelperOutput output)
    {
        output.Attributes.SetAttribute("target", "_blank");
        output.Attributes.SetAttribute("rel", "noopener noreferrer");
        output.Attributes.RemoveAll("dis-baglanti");   // kendi izini sil
    }
}
```

```
<a href="https://learn.microsoft.com" dis-baglanti>Learn</a>
```

Tag Helper mi, View Component mi?

Hafta 3'te View Component'i öğrenmiştik. İkisi de "parça üreten" yapılar ama işleri farklı:

	Tag Helper	View Component
Ne üretir	**Etiketin kendisini** dönüştürür	Bağımsız bir **view parçası**
Kendi view'ı var mı	Hayır (kodla yazar)	**Evet** (`Default.cshtml`)
Veriyi kim getirir	Çağıran	**Kendisi** (servis enjekte edebilir)
Tipik iş	`asp-for` , `asp-action` gibi attribute'ler	Menü, rozet, sepet özeti

Kaba kural: **attribute düzeyinde bir dönüşüm** mü yapıyorsunuz → Tag Helper. **Kendi verisi olan bir blok** mu basıyorsunuz → View Component.

---

2. Sayfa Düzeyindeki Tag Helper'larda Ayrıntılar

Ders notu bölüm 1.6'da her birinin ne ürettiğini ölçtük. Burada gerçek bir projede karşınıza çıkacak hâlleri var.

Gerçek bir CDN ile `asp-fallback-*`

Örnek projede asıl adresi bilerek yanlış verdik. Gerçek kullanım şöyledir:

```
<link rel="stylesheet"
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      asp-fallback-href="~/lib/bootstrap/dist/css/bootstrap.min.css"
      asp-fallback-test-class="visually-hidden"
      asp-fallback-test-property="position"
      asp-fallback-test-value="absolute"
      crossorigin="anonymous" />
```

Sınama sınıfı, asıl dosyanın içinde **bulunduğundan emin olduğunuz** bir sınıf olmalıdır (Bootstrap 5'te `visually-hidden`). CDN dosyası için `integrity` değeri de verirseniz Tag Helper aynı değeri yedek bağlantıya da kopyalar (Learn'deki çıktıda görülür); yerel kopya CDN'dekiyle birebir aynı değilse yedek de yüklenmez. Bunu `asp-suppress-fallback-integrity="true"` kapatır.

`<environment>` ile birlikte sık görülen kalıp: geliştirmede yerel ve okunabilir dosya, yayında CDN'den küçültülmüş dosya.

```
<environment include="Development">
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.css" />
</environment>
<environment exclude="Development">
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/…/bootstrap.min.css" … />
</environment>
```

⚠️ Production'ı yerelde (`dotnet run -e ASPNETCORE_ENVIRONMENT=Production`) denediğinizde `KampusKitap.styles.css` **500** verir: derleme klasöründeki bu dosya yalnızca Development'ta sunulur. `dotnet publish` ile yayımlanan uygulamada sorun yoktur. (Ölçüldü.)

`<cache>` ve `<distributed-cache>`

Attribute	Ne yapar
`expires-after`	İlk çizimden bu kadar sonra kopya silinir
`expires-sliding`	Son kullanımdan bu kadar sonra silinir
`expires-on`	Belirli bir anda silinir
`vary-by-query`, `vary-by-route`, `vary-by-cookie`, `vary-by-header`, `vary-by-user`, `vary-by-culture`	Bu değerlerin her biri için ayrı kopya
`enabled`	`false` ise önbellek devre dışı

`<distributed-cache>` aynı attribute'leri taşır; farkı, anahtarını sayfadaki konumdan değil `name` attribute'ünden alması ve kopyayı `Program.cs`'te kaydedilen `IDistributedCache` deposuna (Redis, SQL Server) yazmasıdır. Depo kaydedilmemişse o da `<cache>`'in bellek deposunu kullanır.

`asp-area` ve `asp-page`

Bu derste Area ve Razor Pages kullanmıyoruz; Learn'deki yazımlar ve çıktıları:

```
<a asp-area="Blogs" asp-controller="Home" asp-action="AboutBlog">About Blog</a>
<!-- /Blogs/Home/AboutBlog -->

<a asp-page="/Attendee" asp-route-attendeeid="10">View Attendee</a>
<!-- /Attendee?attendeeid=10 -->
```

`asp-page` verilen sayfa yoksa Tag Helper **o anki sayfanın** adresini üretir ve uyarı vermez — ders notu bölüm 2.5'teki "adres üretildi diye çalışır demek değildir" dersinin bir başka yüzü.

---

3. `[AutoValidateAntiforgeryToken]` ve AJAX

Tek tek yazmak yerine

Ders notu bölüm 4.3'te her POST action'ına `[ValidateAntiForgeryToken]` yazdık. Unutmak mümkün — ve unutulduğunda hiçbir uyarı yok.

Daha güvenli varsayılan, **filtreyi uygulamanın tamamına** uygulamaktır:

```
builder.Services.AddControllersWithViews(secenekler =>
{
    secenekler.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
});
```

`[AutoValidateAntiforgeryToken]` **GET, HEAD, OPTIONS ve TRACE dışındaki** bütün isteklerde token arar. Yani yazmayı unuttuğunuz action da korunur.

Muaf tutmak istediğiniz bir action olursa:

```
[IgnoreAntiforgeryToken]
public IActionResult WebhookAl() { … }
```

> ⚠️ Bu ayarı **projenizin başında** yaparsanız, Hafta 4'ten kalan elle yazılmış formlarınız 400 vermeye başlar. Projede bunu bilerek yapmadık: Hafta 4'ün deneyleri çalışır durumda kalsın diye yalnızca `KitapController.EkleTH` korumaya alındı.

AJAX isteklerinde token

Form göndermeyen, JavaScript'ten POST eden sayfalarda token'ı **siz** taşırsınız. Önce sayfaya bir token bastırın:

```
@inject Microsoft.AspNetCore.Antiforgery.IAntiforgery antiforgery
@{ var token = antiforgery.GetAndStoreTokens(Context).RequestToken; }
<script>const TOKEN = "@token";</script>
```

Sonra başlıkla gönderin:

```
fetch('/Kitap/Sil/3', {
    method: 'POST',
    headers: { 'RequestVerificationToken': TOKEN }
});
```

Çatının o başlığı tanıması için:

```
builder.Services.AddAntiforgery(o => o.HeaderName = "RequestVerificationToken");
```

---

4. `SameSite` Cookie Attribute'ünün Katkısı

Ders notu bölüm 4.1'de saldırının temeli şuydu: **tarayıcı cookie'yi otomatik ekliyor.** `SameSite` attribute'ü tam olarak bu davranışı sınırlar.

`SameSite` değeri	Cookie ne zaman gönderilir
`Strict`	Yalnızca **aynı siteden** gelen isteklerde
`Lax`	Aynı siteden + **üst düzey GET gezinmelerinde** (bağlantıya tıklama)
`None`	Her zaman — ama `Secure` zorunludur

Modern tarayıcılarda varsayılan `Lax`'tır. `Lax`, başka bir siteden gelen **POST** isteğine cookie **eklemez** — yani ders notundaki saldırı senaryosunu büyük ölçüde zaten engeller.

O hâlde token'a hâlâ gerek var mı? Evet:

- `SameSite` bir **tarayıcı** korumasıdır; eski ya da ayarı değiştirilmiş bir tarayıcıda yoktur.
- Saldırı aynı siteden de gelebilir (örneğin kullanıcı içeriği barındıran bir alt alan adı).
- Güvenlik **katmanlıdır**: bir katmanın varlığı diğerini gereksiz kılmaz.

> Cookie'lerin tamamı — `Expires`, `HttpOnly`, `Secure`, `SameSite`, `Path` — **Hafta 7**'nin konusudur.

---

5. API'lerde CSRF Neden Farklı Ele Alınır?

CSRF'in çalışma şartı, kimlik bilgisinin **tarayıcı tarafından otomatik eklenmesidir**. Cookie böyledir. Ama:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs…
```

Bu başlığı tarayıcı otomatik eklemez; onu **JavaScript kodunuz** ekler. Saldırganın sayfasından yapılan istek bu başlığı taşımaz — dolayısıyla CSRF riski **doğmaz**.

Kimlik taşıyıcı	Otomatik gider mi	CSRF riski

Cookie	**Evet**	**Var** → token gerekir
`Authorization` başlığı (JWT)	Hayır	Yok

Bu yüzden Web API'lerde anti-forgery token genellikle **kullanılmaz**. Ama API'niz kimlik için **cookie** kullanıyorsa risk geri gelir.

> Web API ve JWT **Hafta 12**'nin konusudur.

---

6. Doğrulama Mesajlarını Türkçeleştirmek

Üretilen HTML'de mesajlar İngilizce:

```
data-val-required="The Başlık field is required."
```

Alan **adı** Türkçe (çünkü `[Display]` ile biz verdik), ama cümle kalıbı çatıdan geliyor. Üç yol var:

1. Mesajı attribute'te yazmak (en basit, Hafta 6'nın konusu):

```
[Required(ErrorMessage = "{0} alanı zorunludur.")]
[Display(Name = "Başlık")]
public string Baslik { get; set; } = "";
```

2. Kaynak dosyası (`.resx`) kullanmak — çok dilli projeler için doğru yol:

```
builder.Services.AddControllersWithViews()
    .AddDataAnnotationsLocalization();
```

3. Kültürü sabitlemek — ondalık ayracı sorununu da çözer:

```
var kultur = new CultureInfo("tr-TR");
app.UseRequestLocalization(new RequestLocalizationOptions
{
    DefaultRequestCulture = new RequestCulture(kultur),
    SupportedCultures = [kultur],
    SupportedUICultures = [kultur]
});
```

> Üçüncü maddenin `decimal` üzerindeki etkisi için `H04-detay.md` bölüm 5'e bakın.

---

7. `asp-for` İfadesi Nasıl Çözülüyor?

`asp-for="Baslik"` tırnak içinde **metin gibi** görünür ama değildir. Çatı onu `ModelExpression` tipinde bir ifadeye çevirir — yani derleyici onu **gerçek bir özellik erişimi** olarak görür.

Bu yüzden:

- `asp-for="Balsik"` → **derleme hatası** (`CS1061`).
- `asp-for` kullanılan view'da `@model` yoksa model `dynamic` olur ve ifade ağacı kurulamaz → **`CS1963`**.
- İç içe erişim (`asp-for="Adres.Sehir"`) ve indis (`asp-for="Satirlar[0].Ad"`) aynı mekanizmayla çalışır.

`name` üretimi de buradan gelir: ifadenin **yolu** `name`'e yazılır. Hafta 4'te Model Binding'in `Adres.Sehir` biçimini beklediğini söylemiştik; `asp-for` tam olarak o biçimi üretir. İki hafta, aynı sözleşmenin iki ucu.

---

8. Daha İleri Gitmek İsteyenlere

Konu	Nereden başlanır
Tag Helper Components (sayfaya her view'da içerik enjekte etmek)	`ITagHelperComponent`
`[HtmlAttributeNotBound]` ve `[ViewContext]`	Tag Helper içinden `ViewContext`'e erişmek
`Order` özelliği	Aynı etikete birden çok Tag Helper bağlandığında sıra
`asp-validation-for` / `asp-validation-summary`	**Hafta 6**
`IHtmlGenerator`	Çatının `asp-for` için kullandığı alt katman

Kaynak: Author Tag Helpers